

Lab 05

L0501 ให้นักศึกษาเขียนโปรแกรมเพื่อทดลองการใช้งานชุดคำสั่งที่มีการใช้ `while` ซึ่งโปรแกรมจะจำลองการเคลื่อนที่ของยานพาหนะที่มีการเคลื่อนที่จากหยุดนิ่ง ผ่านการใช้สูตร $v = u + at$

เมื่อ v คือความเร็วปลาย, u คือความเร็วต้น, a คือความเร่ง, และ t คือเวลา โปรแกรมจะทำการรับค่าความเร่ง (a) จากผู้ใช้โปรแกรม และจะทำการคำนวณค่า v ในแต่ละช่วงเวลา t กำหนดให้ u เป็น 0 (จากหยุดนิ่ง) โปรแกรมจะทำการหยุดการคำนวณค่า v เมื่อพบว่าค่า v ที่ได้สุดท้ายมีค่าตั้งแต่ 20 ขึ้นไป

ตัวอย่างผล

```
Starting the engine
Enter the acceleration value (m/s^2): 7.321
Current speed at t=0 is 0.000 m/s
Current speed at t=1 is 7.321 m/s
Current speed at t=2 is 14.642 m/s
Current speed at t=3 is 21.963 m/s
```

```
Starting the engine
Enter the acceleration value (m/s^2): 11.23
Current speed at t=0 is 0.000 m/s
Current speed at t=1 is 11.230 m/s
Current speed at t=2 is 22.460 m/s
```

ผลลัพธ์

The screenshot shows a C program in a code editor with a 'Run' button. The code implements a while loop that calculates speed v at each time step t until v reaches or exceeds 20.0. The output window displays the results of the program execution.

```
main.c Run Output
1 #include <stdio.h>
2 int main() {
3     float a;
4     float u = 0.0;
5     float v = 0.0;
6     int t = 0;
7     printf("starting the engine \n");
8     printf("enter the acceleration value (m/s^2): ");
9     scanf("%f", &a);
10    while (v < 20.0) {
11        v = u + (a * t);
12        printf("Current speed at t=%d is v=%.3f m/s\n", t, v);
13        t++;
14    }
15
16    printf("\n");
17
18    return 0;
19 }
```

```
starting the engine
enter the acceleration value (m/s^2): 11.23
Current speed at t=0 is v=0.000 m/s
Current speed at t=1 is v=11.230 m/s
Current speed at t=2 is v=22.460 m/s

=== Code Execution Successful ===
```

L0502 ให้นักศึกษาเขียนโปรแกรมเพื่อทดลองการใช้งานชุดคำสั่งที่มีการใช้ do-while โปรแกรมที่กำหนดจะเป็นโปรแกรมที่มีการรับข้อมูลเลขจำนวนเต็มจากผู้ใช้โดยไม่จำกัดจำนวน และโปรแกรมจะหยุดรับข้อมูลเมื่อผู้ใช้ป้อนเลขติดลบ

ตัวอย่างผล

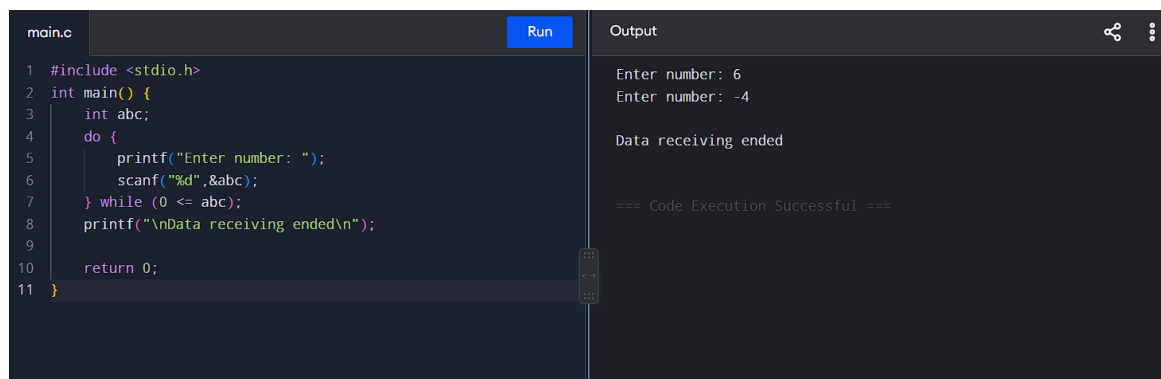
```
Enter number: 4
Enter number: 2
Enter number: -6

Data receiving ended
```

```
Enter number: 6
Enter number: -3

Data receiving ended
```

ผลลัพธ์



```
main.c Run Output
1 #include <stdio.h>
2 int main() {
3     int abc;
4     do {
5         printf("Enter number: ");
6         scanf("%d",&abc);
7     } while (0 <= abc);
8     printf("\nData receiving ended\n");
9
10    return 0;
11 }
```

```
Enter number: 6
Enter number: -4

Data receiving ended

=== Code Execution Successful ===
```

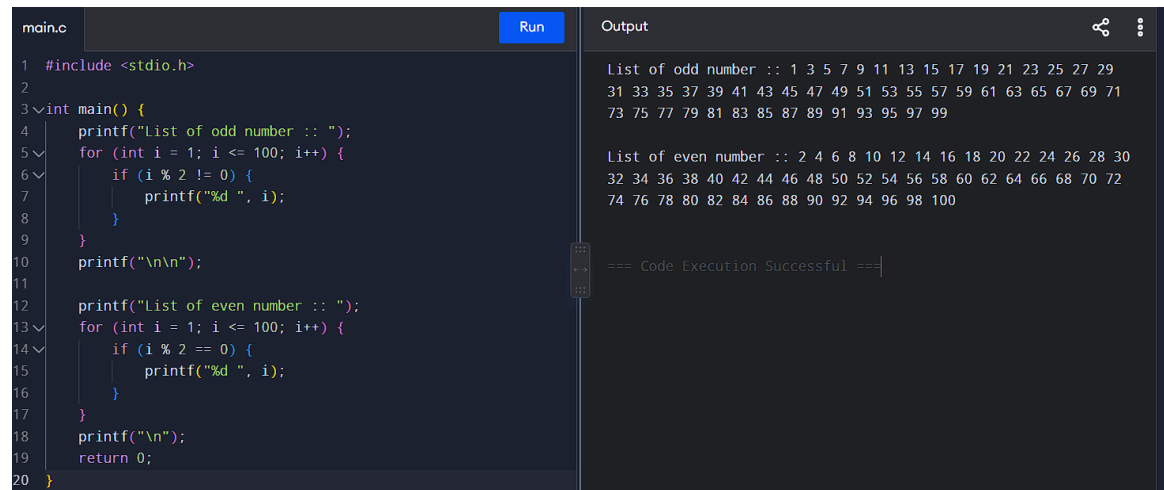
L0503 ให้นักศึกษาเขียนโปรแกรมเพื่อทดลองการใช้งานชุดคำสั่งที่มีการใช้ for โดยโปรแกรมจะทำการแสดงผลชุดเลขคี่ และชุดเลขคู่ตั้งแต่ 1-100 ดังตัวอย่าง

ตัวอย่างผล

```
List of odd number :: 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35
37 39 41 43 45 47 49 51 53 55 57 59 61 63 65 67 69 71 73 75 77 79 81 83
85 87 89 91 93 95 97 99
```

```
List of even number :: 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34
36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82
84 86 88 90 92 94 96 98 100
```

ผลลัพธ์



The screenshot shows a C program in a code editor with a 'Run' button. The code is as follows:

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     printf("List of odd number :: ");
5     for (int i = 1; i <= 100; i++) {
6         if (i % 2 != 0) {
7             printf("%d ", i);
8         }
9     }
10    printf("\n\n");
11
12    printf("List of even number :: ");
13    for (int i = 1; i <= 100; i++) {
14        if (i % 2 == 0) {
15            printf("%d ", i);
16        }
17    }
18    printf("\n");
19    return 0;
20 }
```

The output window displays the following results:

```
Output
List of odd number :: 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29
31 33 35 37 39 41 43 45 47 49 51 53 55 57 59 61 63 65 67 69 71
73 75 77 79 81 83 85 87 89 91 93 95 97 99

List of even number :: 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30
32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72
74 76 78 80 82 84 86 88 90 92 94 96 98 100

=== Code Execution Successful ===
```

L0504 ให้นักศึกษาเขียนโปรแกรมเพื่อแบ่งพื้นที่รูปสี่เหลี่ยมเป็นจำนวน n พื้นที่ โดยโปรแกรมจะรับค่าความกว้าง ความยาว และจำนวน n จากผู้ใช้งาน ถ้าค่าใดค่าหนึ่งใน 3 ค่านี้เป็น 0 หรือติดลบให้แจ้งผู้ใช้งานว่า ติดลบให้แจ้งผู้ใช้งานว่า Error input แล้วรับค่าทั้ง 3 ใหม่อีกครั้ง จนกว่าค่าทั้ง 3 จะมีค่ามากกว่า 0 เมื่อรับค่าเสร็จแล้วให้แสดงผลความกว้าง, ความยาว, และขนาดพื้นที่หลังแบ่งแล้ว ถ้าความกว้างมีค่ามากกว่าความยาว ให้ทำการสลับค่าของตัวแปรระหว่างความกว้างและความยาวก่อนการแสดงผลด้วย

ตัวอย่างผลลัพธ์

```
Enter width: 33.125
Enter length: 21.5
Enter number of areas (n): 4

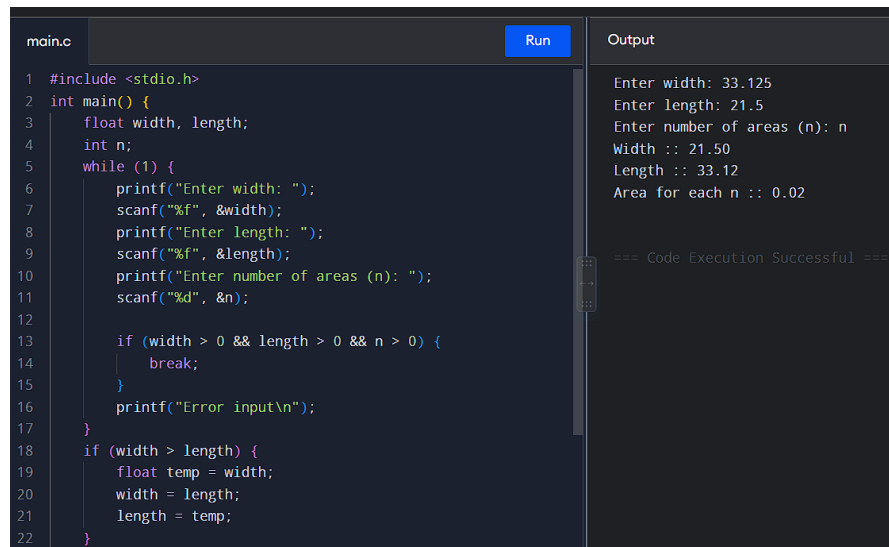
Width :: 21.50
Length :: 33.13
Area for each n :: 178.05
```

```
Enter width: 0
Enter length: 12.3
Enter number of areas (n): 2
Error input

Enter width: 5.67
Enter length: 6.78
Enter number of areas (n): 3

Width :: 5.67
Length :: 6.78
Area for each n :: 12.81
```

ผลลัพธ์



The screenshot shows a C program in a code editor with a 'Run' button. The code calculates the area of a rectangle divided into n equal parts. It includes input validation for width, length, and n . If any value is non-positive, it prints 'Error input' and loops back for input. If width is greater than length, it swaps them. The output shows the program running successfully with the same inputs as the examples above.

```
main.c Run Output
1 #include <stdio.h>
2 int main() {
3     float width, length;
4     int n;
5     while (1) {
6         printf("Enter width: ");
7         scanf("%f", &width);
8         printf("Enter length: ");
9         scanf("%f", &length);
10        printf("Enter number of areas (n): ");
11        scanf("%d", &n);
12
13        if (width > 0 && length > 0 && n > 0) {
14            break;
15        }
16        printf("Error input\n");
17    }
18    if (width > length) {
19        float temp = width;
20        width = length;
21        length = temp;
22    }
```

```
Enter width: 33.125
Enter length: 21.5
Enter number of areas (n): n
Width :: 21.50
Length :: 33.12
Area for each n :: 0.02

=== Code Execution Successful ===
```

L0505 ให้นักศึกษาเขียนโปรแกรมเพื่อคำนวณค่าต่าง ๆ ของนักศึกษาจากคะแนนที่ผู้ใช้ป้อน โดยโปรแกรมจะสามารถรับข้อมูลได้ไม่จำกัดจำนวนของนักศึกษา ซึ่งการรับข้อมูลจะสิ้นสุดเมื่อผู้ใช้โปรแกรมป้อนคะแนนที่มีค่าติดลบเข้ามา สำหรับค่าที่จะมีการคำนวณมีดังนี้

1. คะแนนเฉลี่ย
2. คะแนนสูงสุด
3. คะแนนต่ำสุด
4. นักศึกษาคนใดคือคนที่ได้คะแนนสูงสุด
5. นักศึกษาคนใดคือคนที่ได้คะแนนต่ำสุด

ทั้งนี้การคำนวณค่าทั้งหมด ไม่นับค่าของคะแนนที่ผู้ใช้ป้อนค่าติดลบ

เนื่องจาก Lab 05 ยังไม่ได้มีการเรียนรู้เรื่อง Array จึง **ห้าม ใช้ Array** ในการทำข้อ L0505 โดย **เด็ดขาด ผ่าฝืน -1 จากคะแนนลบในครั้งนี้**

ตัวอย่างผลลัพธ์

```
Student score calculator
```

```
Enter score for student 1: 45
Enter score for student 2: 64
Enter score for student 3: 90
Enter score for student 4: 103
Enter score for student 5: 20
Enter score for student 6: 4
Enter score for student 7: -1
```

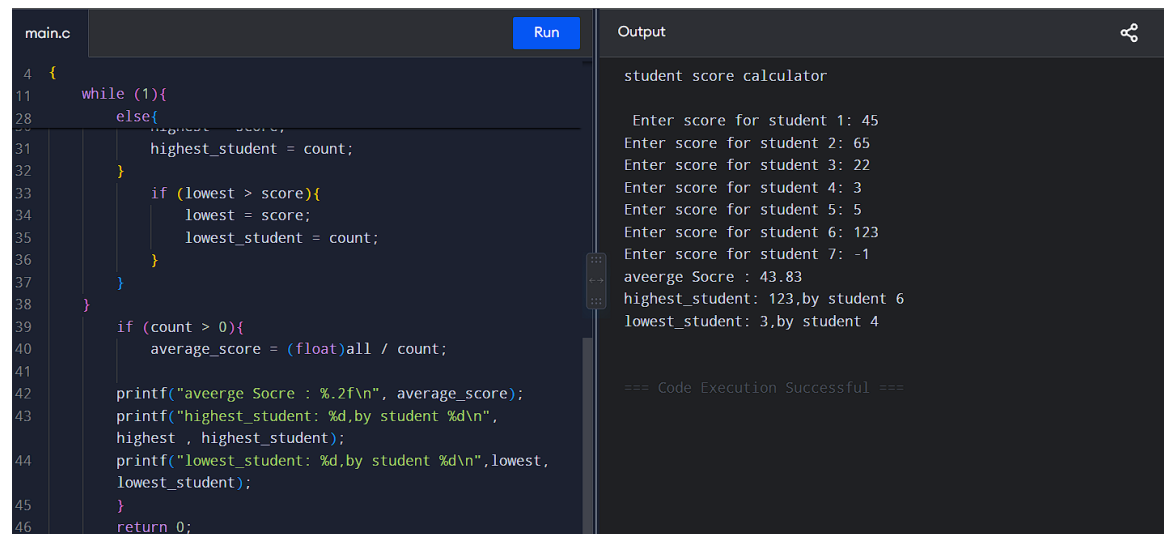
```
Average score :: 54.33
Highest score :: 103, by student 4
Lowest score  :: 4, by student 6
```

```
Student score calculator
```

```
Enter score for student 1: 33
Enter score for student 2: 48
Enter score for student 3: 90
Enter score for student 4: -2
```

```
Average score :: 57.00
Highest score :: 90, by student 3
Lowest score  :: 33, by student 1
```

ผลลัพธ์



The screenshot displays a C program in a code editor with a dark theme. The code is for a 'student score calculator' and is contained within a file named 'main.c'. The code includes a loop to input scores for 7 students, calculates the average score, and identifies the highest and lowest scores. The output window on the right shows the program's execution, including the prompts for student scores, the calculated average score (43.83), and the highest and lowest scores (123 for student 6 and 3 for student 4). The execution is successful.

```
main.c Run Output
4 {
11 while (1){
28     else{
31         highest_student = count;
32     }
33     if (lowest > score){
34         lowest = score;
35         lowest_student = count;
36     }
37 }
38 }
39 if (count > 0){
40     average_score = (float)all / count;
41
42     printf("aveerge Socre : %.2f\n", average_score);
43     printf("highest_student: %d,by student %d\n",
44           highest , highest_student);
45     printf("lowest_student: %d,by student %d\n",lowest,
46           lowest_student);
47 }
48 return 0;
```

student score calculator

Enter score for student 1: 45
Enter score for student 2: 65
Enter score for student 3: 22
Enter score for student 4: 3
Enter score for student 5: 5
Enter score for student 6: 123
Enter score for student 7: -1
aveerge Socre : 43.83
highest_student: 123,by student 6
lowest_student: 3,by student 4

=== Code Execution Successful ===

L0506 ให้นักศึกษาเขียนโปรแกรมคำนวณค่า Factorial ของตัวเลขที่ผู้ใช้ป้อนเข้าสู่โปรแกรม จากนั้นแสดงผลพร้อมวิธีทำ

ตัวอย่าง Factorial เช่น $4! = 4 * 3 * 2 * 1$ เป็นต้น

Hint: ใช้ for น่าจะช่วยให้ง่ายขึ้น

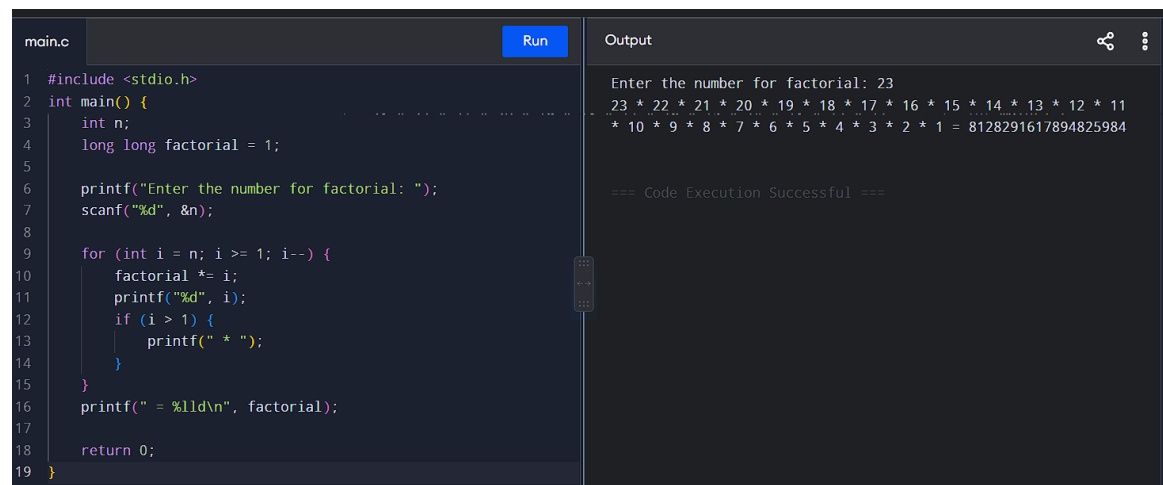
ตัวอย่างผลลัพธ์

```
Enter the number for factorial: 7
7 * 6 * 5 * 4 * 3 * 2 * 1 = 5040
```

```
Enter the number for factorial: 5
5 * 4 * 3 * 2 * 1 = 120
```

```
Enter the number for factorial: 3
3 * 2 * 1 = 6
```

ผลลัพธ์



```
main.c Run Output
1 #include <stdio.h>
2 int main() {
3     int n;
4     long long factorial = 1;
5
6     printf("Enter the number for factorial: ");
7     scanf("%d", &n);
8
9     for (int i = n; i >= 1; i--) {
10         factorial *= i;
11         printf("%d", i);
12         if (i > 1) {
13             printf(" * ");
14         }
15     }
16     printf(" = %lld\n", factorial);
17
18     return 0;
19 }
```

```
Enter the number for factorial: 23
23 * 22 * 21 * 20 * 19 * 18 * 17 * 16 * 15 * 14 * 13 * 12 * 11
* 10 * 9 * 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 8128291617894825984

=== Code Execution Successful ===
```

L0507 ให้นักศึกษาเขียนโปรแกรมคำนวณผลรวมของค่า Factorial ตั้งแต่ 1 จนถึงค่าที่ผู้ใช้ป้อนเข้ามา โดยให้แสดงผลของ Factorial ในแต่ละค่าพร้อมวิธีทำ และแสดงผลรวมสุดท้ายพร้อมวิธีทำ ดังตัวอย่าง

ตัวอย่างผลลัพธ์

Enter the highest number for factorial: 6

Factorial results

1! = 1 = 1

2! = 2 * 1 = 2

3! = 3 * 2 * 1 = 6

4! = 4 * 3 * 2 * 1 = 24

5! = 5 * 4 * 3 * 2 * 1 = 120

6! = 6 * 5 * 4 * 3 * 2 * 1 = 720

Summation of factorial results

1 + 2 + 6 + 24 + 120 + 720 = 873

Enter the highest number for factorial: 8

Factorial results

1! = 1 = 1

2! = 2 * 1 = 2

3! = 3 * 2 * 1 = 6

4! = 4 * 3 * 2 * 1 = 24

5! = 5 * 4 * 3 * 2 * 1 = 120

6! = 6 * 5 * 4 * 3 * 2 * 1 = 720

7! = 7 * 6 * 5 * 4 * 3 * 2 * 1 = 5040

8! = 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 40320

Summation of factorial results

1 + 2 + 6 + 24 + 120 + 720 + 5040 + 40320 = 46233

ผลลัพธ์

The screenshot shows a C program named 'main.c' being executed. The code calculates factorials for numbers 1 through 'max_num' (which is 3) and prints the results. It also calculates the summation of these factorials. The output window shows the following text:

```
Enter the highest number for factorial (1-20): 3

Factorial results
1! = 1 = 1
2! = 2 * 1 = 2
3! = 3 * 2 * 1 = 6

Summation of factorial results
1 + 2 + 6 = 9

=== Code Execution Successful ===
```